REFERENCE · GENALPHA-BSS

The Definitive Guide

Architecture, modules, and how to run them — whole, or one piece at a time. The third book in the set: The Honest Machine tells how it was built and why; The Operator's Manual tells an operator which button and which endpoint, by role; ***this book*** *is the engineer's and the adopter's reference — organized by module, so you can read only the chapters for the part you want and stand it up.*

Status — a living book. This is the down-payment: the full outline (the book's shape and every chapter's scope), the orientation, and one **fully written exemplar chapter — Martech & the CDP** — which locks the template. Each remaining chapter is written as its module completes a hardening pass, so the book documents stable behaviour, never a moving target. Chapters are marked WRITTEN or PLANNED below.

Contents

0.1 · What this is

genalpha-bss is a telecom **Business Support System** built as **38 composable ODA components** — Spring Boot microservices, each exposing a TM Forum Open API — behind one gateway, plus a set of channels (storefront, mobile app, B2B and staff consoles, partner desks). Nothing is operator-specific: any OIDC identity provider, any PostgreSQL, any Kafka-protocol broker. Two demo operators run side by side on a single deployment to prove the multi-tenancy is real.

The premise of *this* book is the premise of ODA itself: **you can adopt the whole thing, or one module.** A greenfield operator runs the full stack. An incumbent with its own BSS might take only the martech/CDP module, or only wholesale settlement, and wire it to the estate it already runs. This book is organized so either reader finds what they need: read **Part 1 (Foundations)** once — it is the substrate no module can do without — then read the **module chapter(s)** you actually want. Each module chapter is otherwise self-contained.

0.2 · How to read this book

Every module chapter follows the **same seven-part skeleton**, so you always know where to look:

Why — the problem the module solves and the design stance it takes.
What — the functional capabilities, in an operator's language.
How — the technical shape: components, the TMF APIs it serves, the events it emits and consumes.
Deploy — the compose / Helm subset to run it, *and its dependency floor*.
Configure — the environment variables, seams, and data-as-config that tune it.

Use — the day-to-day, by console tab and by API.

Verify — the end-to-end suites that prove it, so behaviour is never a claim.

The dependency-floor convention. "Adopt one module" is only *true* for some modules. Martech runs standalone (a bridge feeds it a foreign BSS's events); billing structurally needs party + catalog + the event bus. So every module chapter opens with a ▟ Dependency floor box that states, plainly, what the module cannot run without. This is the honest core of "composable" — it tells an integrator exactly what they are buying into. A book that hid it would be selling a lie.

0.3 · Run it now

```
mvn -q package -DskipTests && docker compose build && docker compose up -d
open http://localhost:8080/shop/    # the storefront (guest browsing works)
```

The gateway is `:8080`; Keycloak is `:8085` (admin/admin). Two seed operators run side by side — **genalpha** (EN/EUR, realm `bss`) and **nova** (NO/NOK, realm `nova`). Back office is `/console/` (`demo` / `demo`).

The one discipline. Every capability in this book has an end-to-end Playwright suite in `ops/e2e/`. When behaviour is in doubt, the suite is the specification: `node ops/e2e/<name>_test.js` runs it against the live stack. The **Verify** section of each chapter names the suites that prove it.

M5 · Martech & the CDP

Module family: `insight` (CDP brain) + `campaign` (orchestration & measurement) + `communication` (delivery) · Console workspace: **Growth** · This is the reference book's exemplar chapter — the template every other module chapter follows.

> **Dependency floor.** The martech module needs, at minimum: **the event backbone** (F3 — Kafka; the audience feed and the ticket hand-off both ride it), **a PostgreSQL** (F4 — trait/audience/job stores with RLS), and **identity** (F2 — to scope reads per tenant and authorize the console). It does *not* need this BSS's own catalog, ordering, or billing: it consumes whatever domain events it is fed. On a foreign BSS it runs against the **bss-bridge** (§How), which maps that estate's events onto the martech envelope — so martech is the one module that genuinely stands alone. For *measurement* (attribution) it additionally needs a source of conversion + revenue events; on this stack that's `campaign` consuming `bss.ordering.events`, but a bridged estate can supply its own.

Why

A normal customer-data platform is a *second* database you copy your customers into on a schedule — reverse-ETL out of the BSS, into a marketing cloud, and back. That copy is always stale (research puts reverse-ETL latency at 15–60 minutes, and near-real-time refresh at ~25–50× the cost), always a second identity graph to reconcile, and always a place your consent decisions can drift out of sync. This module takes the opposite stance: **the operational event bus you already run is the audience feed.** An order completes, a bill is issued, a loyalty tier changes — and the trait is *already* there, in the same database, under the same consent. No export, no round-trip, no second copy. The BSS *is* the CDP.

The second stance is **honesty about outcomes**. Marketing tools are structurally tempted to claim credit for revenue that would have arrived anyway.

This module measures with a **holdout** by default and reports **incremental** revenue — and refuses to report lift for any program that kept no control group.

What

CAPABILITY	WHAT IT DOES
BSS-native traits	A live per-party feature store fed straight off domain events — `product=Fibre 500` , `loyaltyTier=gold` , `region=Oslo` , `churnRisk=high` , `monthlySpend=620` . Single-valued traits **replace on change**, so a customer auto-moves between audiences; `product` is multi-valued and **retracts on cancellation**. You never import these; the bus fills them, and a one-shot **backfill** seeds customers who predate the CDP.
Network device (`deviceModel`)	The handset a SIM is *actually* in — OSS/network truth (EIR/device-detection, IMEI→TAC→model), ingested as a trait. Targets "on an iPhone 15" incl. BYOD, distinct from the `product` a customer *bought*. A device swap re-homes them.
Audiences	Saved rule trees (`all` / `any` / `not` over trait/behaviour leaves, numeric operators) resolved to members on demand, across four **populations**: customer · prospect · organization · visitor.
Prospects	Not-yet-customers. A bought list lands *captured but not reachable*; it becomes targetable only when an explicit lawful basis records consent. The gate is absolute.
Activation	Push an audience to an ad platform (Meta Custom Audiences, Google Customer Match) as a SHA-256-hashed, DNC-filtered match list — to *seed* lookalikes or *suppress* paid spend. One connector, many destinations, async jobs.
Visitor retargeting	Anonymous consented browsers, keyed by `visitorId` , resolved by browsing interest — the on-site-personalization / web-retarget population (device-keyed, not the email path).
Social listening	Pull brand mentions, score sentiment, show share-of-mood.
Social care	Inbound DMs triaged for sentiment + support intent; the ones that need a human become TMF621 trouble tickets, event-driven.
Attribution	Per-program and portfolio-wide holdout-vs-treated lift and **incremental** revenue, with a hard no-holdout-no-claim rule.
Marketing preferences	A customer self-serve opt-out (shop → Account) enforced in the send path (in-app + email) before the frequency cap; a one-click, no-login unsubscribe (HMAC) in every marketing message; opt-out also excludes from ad activations.

Landing pages + lead capture	Standalone branded campaign pages (logo/hero/colour/links, sanitized) with a consent-first form; a ticked submit becomes a consented prospect stamped with the campaign — closing ad/email → landing → lead → nurture. Authored in the console.
Interaction record (TMF683)	Every message sent — *including martech campaigns and journeys* — is logged to the omnichannel interaction timeline (who, what, which channel, which system), so a CSR sees the whole contact history on one 360 view and can pick the next best action.

How

COMPONENTS & THE STANDARDS THEY SERVE

SERVICE	PORT	ROLE
`insight`	8119	The CDP brain: trait store, audiences, prospects, activation, social listening + care. Consumes the bus; serves `/insight/v1/*`.
`campaign`	8108	Journeys + campaigns + measurement; owns holdout, conversion attribution, and the portfolio report. Serves `/tmf-api/campaignManagement/v4/*`.
`communication`	8095	TMF681 delivery over the per-tenant ESP seam; emits engagement + suppression events back.
`bss-bridge`	8140	Optional: maps a *foreign* BSS's event shapes onto the martech envelope (`bss.bridge.events`) so the module runs on any estate.

TMF surfaces: TMF688 (the event envelope), TMF620/GA4 (analytics audiences), TMF621 (trouble tickets, via the care hand-off), TMF699 (social lead → sales funnel), TMF683 (the interaction record every message writes to).

THE TRAIT FEED — EVENTS IN

`insight`'s `BssTraitListener` subscribes to the operational topics (`bss.ordering.events` , `bss.party.events` , `bss.loyalty.events` , `bss.billing.events` , `bss.intelligence.events` , and — on a foreign estate — `bss.bridge.events`). Each event type maps to one trait branch: a completed order writes `product` holdings; `IndividualCreateEvent` writes `email` + `region` ; `LoyaltyTierChangedEvent` replaces `loyaltyTier` ;

`CustomerBillCreateEvent` writes `monthlySpend`. Adding a trait is one more listener branch, not a new pipeline. Traits land in `party_trait` (RLS by tenant).

AUDIENCE RESOLUTION — THE SCALE PATH

A trait-only *customer* audience compiles to **one set-based SQL query** over the indexed `party_trait` table (leaf → `SELECT`; `all` → `INTERSECT`; `any` → `UNION`; `not` → all-parties `EXCEPT` child) — this carries to millions of rows on Postgres alone. Behavioural trees (browsing interest, GA4) keep an in-memory path; prospects, organizations and visitors have dedicated resolvers. `GET /audience/{id}/members?explain=true` reports which path ran (`sql · memory · prospect · visitor · organization`). Materialized snapshots (`POST /audience/{id}/refresh`) freeze a set for instant reads; a bounded scheduler re-materializes the stalest.

ACTIVATION — EVENTS OUT, TO THE AD PLATFORMS

The connector is an `AdDestination` interface with per-platform implementations (`MetaDestination` → `schema:[EMAIL_SHA256]` / `data`; `GoogleDestination` → `operations:[{create:{hashed_email}}]`). Emails are hashed once (`Hashing.sha256`), the do-not-contact ledger is filtered out, and the list is pushed in batches. Activation is **asynchronous**: `POST /audience/{id}/activate {externalAudienceId, mode, destination}` returns `202 {jobId, status:"queued"}` and the export runs under `TenantContext.actAs`; poll `GET /audience/activation/{jobId}` for `queued → running → done`.

SOCIAL CARE — THE EVENT-DRIVEN TICKET HAND-OFF

The clean pattern to study: `insight` has *no* ticket-write scope, and shouldn't. `SocialCareService.sync` scores each DM and, for the ones needing a human, **emits** `SocialCareTicketRequested` on `bss.insight.events`. `trouble-ticket`'s `SocialCareTicketListener` consumes it and opens a `socialCare` ticket inside `TenantContext.actAs(tenantId)` — the RLS datasource reads that thread-local and sets the Postgres session tenant, so the consumer writes correctly-scoped rows with no cross-service auth on the hot path. Idempotent per source DM (dedup on `relatedEntity`), so an at-least-once re-delivery never double-opens a case.

ATTRIBUTION — THE HONEST NUMBER

Each campaign/journey keeps its own holdout-vs-treated measurement. `AttributionService.report()` (behind `GET /campaign/attribution`) rolls

every program up: per-program lift, a blended portfolio, and **incremental revenue = (treated-per-head − holdout-per-head) × heads reached**. A program with `heldOut == 0` reports reach and gross but its incremental is `null` — you cannot measure a lift you never left room to see.

Deploy

Full stack: the module is already in `docker compose up -d`. To run **martech alone against a foreign BSS**, the minimal set is:

```
docker compose up -d \
  postgres kafka keycloak gateway \   # F2/F3/F4 foundations
  insight campaign communication \    # the module
  bss-bridge mock-social mock-esp     # foreign-event adapter + dev seams
```

Point the bridge at the source estate (map its event shapes in `BridgeService`), and insight consumes the normalized events with zero martech-service changes. On Kubernetes the Helm chart's per-component toggles let you install just these subcharts; see F5.

Configure

VARIABLE (SERVICE)	SETS
`BSS_INSIGHT_TRAIT_TOPICS` (insight)	Which event topics become traits — add `bss.bridge.events` for a foreign estate.
`SOCIAL_API_URL` / `SOCIAL_ACCESS_TOKEN` / `SOCIAL_ACCOUNT_ID` (insight)	The Meta-shaped platform + brand handle for activation, listening, care.
`GOOGLE_API_URL` / `GOOGLE_ACCESS_TOKEN` (insight)	The Google Customer Match destination (production: OAuth from the tenant's secret store).
`INSIGHT_REFRESH_ENABLED` / `_INTERVAL_MS` / `_MAX_PER_RUN` (insight)	The bounded snapshot auto-refresh scheduler.
`FREQUENCY_CAP_MAX` / `FREQUENCY_CAP_WINDOW_HOURS` (communication)	The per-tenant send guardrail every message obeys.
`BSS_SOCIAL_CARE_SOURCE_TOPIC` (trouble-ticket)	Where the ticket service listens for care requests (default `bss.insight.events`).

Data-as-config, not env: audiences, holdout %, conversion window, and A/B arms are authored in the console (or via the APIs) and stored as rows — no redeploy to change a campaign.

Use

Everything lives in the console under **Growth**. The step-by-step operator walkthrough is the Martech & CDP guide; in brief:

TASK	CONSOLE TAB / API
Build & resolve audiences	Growth → Audience builder · Saved audiences · `POST /insight/v1/audience`
Import prospects (consent-gated)	Audience builder (population = Prospects) · `POST /insight/v1/prospect/import`
Push to ad platforms	Saved audiences → Activate · `POST /insight/v1/audience/{id}/activate`
Lift & incremental revenue	Growth → **Attribution** · `GET /campaign/attribution`
Brand mentions + mood	Growth → Social listening · `POST /insight/v1/listening/sync`
Inbound DMs → tickets	Growth → Social care · `POST /insight/v1/care/sync`
Author landing pages	Growth → **Landing pages** · `POST /insight/v1/landing` ; public `GET …/{slug}/view` + `POST …/{slug}/lead`
Marketing opt-out (customer)	shop → Account → Marketing preferences · `GET/POST /tmf-api/communicationManagement/v4/marketingPreference` ; one-click `GET /esp/v1/unsubscribe`
Full contact history (CSR)	CSR 360 timeline · `GET /tmf-api/partyInteraction/v4/partyInteraction?relatedPartyId=` — every send incl. martech, one omnichannel record
Read the consent ledger	Console → **Privacy & governance → Visitor consent** · `GET /insight/v1/profile?offset=&limit=&q=` — paginated + searchable-by-id (X-Total-Count), read-only; who is tracked, under which consent, and what was learned

One customer timeline (TMF683). Every message the module sends — campaign, journey step, or system notice — publishes `CommunicationMessageCreateEvent`, which the `party-interaction` service records as a touchpoint (who · what · channel · sending system, deduped by event id). So marketing isn't a silo: a CSR sees marketing + service messages together on the customer's 360, which is exactly what next-best-action reads. (Honest limits today: the touchpoint carries the subject + channel, not the specific campaign name; opens/clicks and inbound social DMs are separate records, not timeline touchpoints.)

Two kinds of consent, kept apart. The Visitor-consent tab's flags — `analyticsConsent` (gates storing breadcrumbs) and `personalizationConsent` (gates using them on-site) — are **first-party tracking/personalization** consent, the cookie-banner family. Permission to *contact* someone is a **separate** consent: a prospect's `consent` + `lawfulBasis` and the do-not-contact suppression ledger, filtered at every send and activation. The law treats them differently, and so does the system. Reading the Insight tab and the tracking-vs-contact distinction is walked through in the Martech & CDP guide §8.

Verify

The suites that are the specification for this module:

`audience_native_test` · `audience_sql_test` · `audience_snapshot_test` · `audience_scheduler_test` · `audience_activation_test` · `audience_async_activation_test` · `connector_multidest_test` · `visitor_retargeting_test` · `traits_region_org_test` · `product_trait_retraction_test` · `device_model_test` · `frequency_dnc_test` · `engagement_test` · `marketing_preference_test` · `prospect_reach_test` · `social_listening_test` · `social_publish_test` · `social_lead_test` · `social_care_test` · `landing_page_test` · `attribution_report_test` · `bss_bridge_test` (the "runs on any BSS" proof).

Read this chapter as the template. Every module chapter that follows — billing, ordering, wholesale, and the rest — is written to this same seven-part shape, opens with its own dependency-floor box, and ends with the suites that prove it. When a module finishes its hardening pass, its chapter is a fill-in, not a fresh design.

Planned chapters

The chapters below are outlined and will be written to the M5 template as each module completes its hardening pass. Until then, the *Operator's Manual* and `docs/architecture.md` cover this ground operationally; this book will add the module-axis, why→verify treatment.

Part 1 · Foundations

F1 Gateway & routing · **F2 Identity, tenancy & isolation** · **F3 The event backbone** · **F4 Data & migrations** · **F5 Deployment** · **F6 Observability & operations**. These are read once and depended on by every module chapter's dependency floor.

Part 2 · The other modules

M1 Product catalog · **M2 Ordering & fulfilment** · **M3 Billing & revenue** · **M4 Party, accounts & identity** · **M6 Assurance & care** · **M7 Wholesale & open access** · **M8 AI & the digital workforce** · **M9 Channels** · **M10 Knowledge, loyalty & recommendations**. Each: Why · What · How · Deploy (+ dependency floor) · Configure · Use · Verify.

Part 3 · Cross-cutting & appendices

X1 Security · **X2 Privacy & GDPR** · **X3 Scale & load** · **X4 Standards & conformance** · **A** Seam catalog · **B** Environment reference · **C** Glossary (→ tmf-reference.md).